INFOMCV Assignment 2

M. Jim 1511203, L.M.M. Laan 6125573, Group number 20

March 4, 2026

1 Explaination of methods

Our 3D reconstruction pipeline consists of three main stages: background subtraction, post-processing, and voxel model construction.

Background Subtraction

Foreground segmentation is performed using a static background subtraction method in HSV color space. A background model is built for each camera by averaging multiple frames from a dedicated background video. This averaging suppresses moving objects while retaining static background information. For each incoming frame, the absolute difference from the background model is computed in the Hue, Saturation, and Value channels. Pixels exceeding a predefined threshold in any channel are classified as foreground.

1.1 Post-Processing

The raw foreground masks often contain noise, small isolated regions, and occasional holes due to imperfect background subtraction. To improve the quality of the segmentation, we first apply morphological opening, which removes small noise pixels and eliminates tiny disconnected regions. This is followed by dilation to fill small gaps and slightly enlarge the foreground areas, helping to recover missing parts of the object. After these morphological operations, we extract only the largest connected component in the mask, under the assumption that it corresponds to the main object in the scene. This ensures that spurious small blobs from shadows or background artifacts are removed, producing a cleaner and more consistent mask for subsequent voxel reconstruction.

Voxel Model Construction

The 3D space is discretized into a regular voxel grid. For each voxel, its projection into all camera images is computed using the camera intrinsic and extrinsic parameters. A voxel is marked as occupied if its projections fall onto foreground pixels in all camera views. The resulting set of occupied voxels represents the reconstructed 3D shape of the object. The voxel coordinates are then converted to the engine coordinate system for visualization or further processing.

2 Extrinsic camera parameters

Camera 1

Rotation matrix R_1:

$$R_1 = \begin{bmatrix} 0.7417 & -0.6707 & -0.0093 \\ 0.0877 & 0.0831 & 0.9927 \\ -0.6650 & -0.7371 & 0.1205 \end{bmatrix}$$

Translation vector t_1 :

$$t_1 = \begin{bmatrix} 215.7067 \\ 892.2991 \\ 4650.9175 \end{bmatrix}$$

Camera 2

Rotation matrix R_2:

$$R_2 = \begin{bmatrix} 0.9990 & -0.0430 & 0.0074 \\ -0.0085 & -0.0266 & 0.9996 \\ -0.0428 & -0.9987 & -0.0270 \end{bmatrix}$$

Translation vector t_2 :

$$t_2 = \begin{bmatrix} -230.2027 \\ 1481.7606 \\ 3577.4638 \end{bmatrix}$$

Camera 3

Rotation matrix R_3:

$$R_3 = \begin{bmatrix} 0.257 & -0.966 & 0.023 \\ 0.006 & 0.025 & 1.000 \\ -0.967 & -0.256 & 0.012 \end{bmatrix}$$

Translation vector t_3:

$$t_3 = \begin{bmatrix} -416.323 \\ 1249.937 \\ 3315.521 \end{bmatrix}$$

Camera 4

Rotation matrix R_4:

$$R_4 = \begin{bmatrix} 0.663 & 0.748 & -0.040 \\ -0.100 & 0.142 & 0.985 \\ 0.742 & -0.649 & 0.169 \end{bmatrix}$$

Translation vector t_4 :

$$t_4 = \begin{bmatrix} -1001.149 \\ 972.636 \\ 4408.260 \end{bmatrix}$$

3 Background subtraction

Foreground segmentation is performed using a static background subtraction approach in HSV color space. For each of the four cameras, a background model is trained from a separate background video sequence. All image processing operations are implemented using the OpenCV library.

3.1 Background Model Estimation

For every camera, 50 frames are sampled evenly across the full background video. If the total number of frames is N, the sampled frame index is computed as

$$\text{idx} = \frac{i \cdot N}{50}$$

Each sampled frame is converted from BGR to HSV color space. HSV is chosen instead of RGB because it separates chromatic information (Hue and Saturation) from brightness (Value), which makes the method more robust to illumination changes.

All sampled HSV frames are accumulated in floating-point precision and averaged pixel-wise:

$$B(x, y) = \frac{1}{K} \sum_{i=1}^{K} F_i(x, y)$$

where K is the number of successfully read frames. This produces a mean background image. The assumption is that moving objects do not occupy the same pixel location consistently across the sampled frames, so averaging suppresses foreground influence and retains static background information.

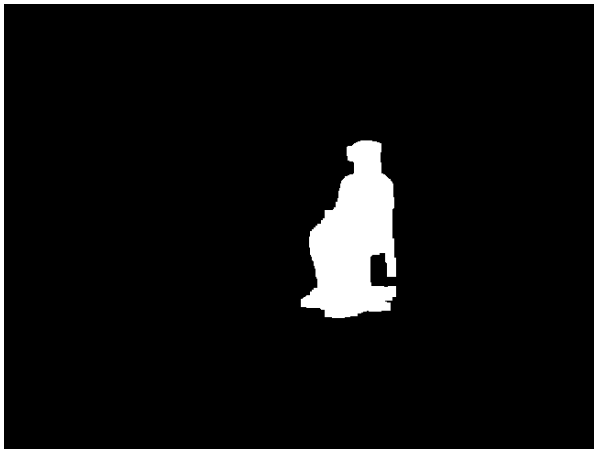

Figure 1: Foreground detection result from Camera 1

3.2 Foreground Classification

For each frame of the input video, the following steps are performed:

1. Convert the frame to HSV color space.
2. Compute the absolute difference between the current frame and the background model.
3. Split the difference image into Hue, Saturation, and Value channels.

A pixel is classified as foreground if at least one of the channel differences exceeds a predefined threshold:

$$Hue > 15 \quad \vee \quad Saturation > 30 \quad \vee \quad Value > 30$$

The thresholds were determined empirically by testing several frames and visually inspecting the resulting foreground masks.

If none of the three conditions is satisfied, the pixel is classified as background. The result is a binary foreground mask.

3.3 Post-Processing

The raw foreground masks contain noise and small isolated regions. To improve the segmentation quality, several morphological operations are applied.

First, a morphological opening operation is performed to remove small noise pixels and isolated foreground regions. After that, a morphological closing operation is applied to reconnect fragmented foreground regions and preserve thin structures such as chair legs or arms that may have been separated during thresholding. Finally, a small dilation step enlarges the foreground regions slightly and fills small gaps.

After the morphological filtering, connected component analysis is performed on the binary mask. Instead of keeping only the single largest blob, the largest connected component is first identified and then slightly dilated. Any additional components that overlap with this expanded region are also retained. This ensures that nearby structures belonging to the same physical object (for example parts of a chair connected to the person) are preserved while still removing small isolated noise regions.

This approach improves robustness compared to selecting only the largest contour, which may remove thin or weakly connected foreground structures.

For each camera (Cam1–Cam4), binary foreground masks are generated and stored as images. White pixels represent detected foreground regions, while black pixels correspond to background. The largest blob selection ensures that only the main moving object remains visible in the final mask.

Figure 2: Foreground detection result from Camera 2

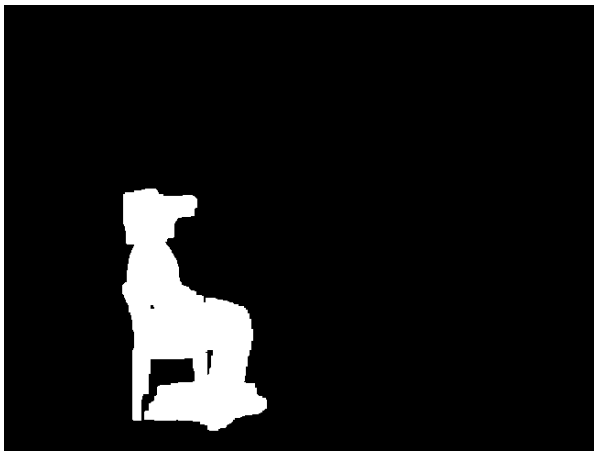

Figure 3: Foreground detection result from Camera 3

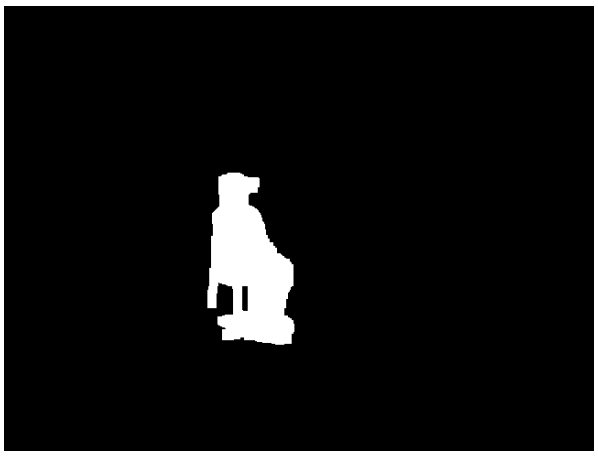

Figure 4: Foreground detection result from Camera 4

4 Choice tasks

4.1 Choice 2

This script calculates the HSV difference thresholds to separate moving foreground objects from a static background. It creates a background model by averaging the HSV values from frames of a video file named background.avi. The script then samples frames from another video file, video.avi, and performs a grid search over potential hue, saturation, and value thresholds. For each set of thresholds, it generates a foreground mask by calculating the absolute difference in HSV values and applying per-channel thresholding, followed by morphological cleanup. The script assesses the quality of the masks without labels using certain heuristics: evaluating the reasonable size of the foreground area, minimizing salt-and-pepper noise, and ensuring temporal stability. The best thresholds are saved and used to create mask images.

4.2 Choice 3

This code assigns RGB colors to a reconstructed voxel using depth-aware visibility. It loads each camera's calibration, builds a voxel grid, and precomputes projections and depths per camera. For each frame, it reconstructs active voxels by intersecting the four silhouette masks. Then, for each camera, it projects active voxels to pixels and groups voxels by pixel. Only the closest voxel per pixel is considered visible. The visible voxel samples the color of the camera image at that pixel. Colors are accumulated across cameras and averaged.

4.3 Choice 5

The code reconstructs a 3D visual hull from four calibrated cameras using a voxel grid. First, camera calibration parameters are loaded from XML files. A 3D grid of voxels representing the capture space is created. Each voxel is projected into every camera image using the calibration parameters, and an inverse lookup table maps image pixels to the voxels that project onto them. For each frame, foreground masks are loaded and used to update which voxels are visible in each camera. Voxels considered in all cameras are kept as part of the reconstruction. The most likely reason the result is incorrect is a mismatch between the voxel grid bounds and the camera coordinate axis convention.

4.4 Choice 6

This script accelerates the code execution through three primary mechanisms. First, it parallelizes the construction of the lookup table: the voxel projections for each camera are computed in separate processes using 'ProcessPoolExecutor'. Second, it parallelizes background subtraction: each camera independently computes an averaged HSV background model from 'background.avi' and then generates foreground masks for all frames of 'video.avi'. Third, the voxel reconstruction is further accelerated by employing one-dimensional mask indices, so that visibility checks are performed via rapid 1D lookups instead of more computationally expensive 2D indexing operations.

4.5 Choice 7

This approac focuses on automated selection and rejection of low-quality calibration frames for camera intrinsics. First, a set of random frames is sampled from the calibration video. Each frame is evaluated for quality: sharpness is measured using the variance of the Laplacian, checkerboard coverage is computed to ensure sufficient area of the board is visible, and corner points too close to the image borders are then rejected. Only frames that pass all of these points are kept. The selected frames are then used to detect checkerboard corners and perform calibration. The extrinsics are extracted via manually clicking the 4 outer corners the same way as in Assignment 1. Then the intrinsics and extrinsics are stored in the config file.

4.6 Link to video

https://www.youtube.com/watch?v=0z6Ps1Ce9SA

5 GenAI prompt

(Mention tool and prompt, and how the output was used. No limit on space, add all prompts you performed.)

References